

HookSuite

Memoria técnica · Práctica 3 — «Del prototipo al producto»

Máster en Ciberseguridad & IA · Evolve Academy

 **HUECO** — responsable: **José María**. Portada: nº de grupo definitivo, integrantes, fecha de entrega, enlace al repo y enlace al vídeo (apartado 1).

1. Portada

Producto: **HookSuite** — herramienta web de auditoría de seguridad «tipo Burp Suite», con IA. Grupo: **nº por confirmar**. Entrega: **16 de octubre de 2026**.

⚠ **HUECO** — responsable: **José María**. Integrantes con rol, enlace al repositorio público y enlace al vídeo (≥10 min). Se rellena al congelar.

2. Resumen ejecutivo

HookSuite resuelve la auditoría web asistida por IA desde el navegador: interceptar y repetir peticiones, fuzzing, utilidades (hash/encoder/regex), análisis de vulnerabilidades y un módulo de IA que prioriza hallazgos. La P3 parte del prototipo de la P1 y lo lleva a un estado ejecutable, seguro y reproducible.

⚠ **HUECO** — responsable: **José María + Claude**. Cerrar a 1 página cuando el producto esté congelado: qué hace, para quién, y en qué estado final se entrega.

3. Punto de partida

HookSuite nació en la Práctica 1 como prototipo funcional de una suite de auditoría web «tipo Burp Suite» operada desde el navegador y asistida por IA. Quedó desplegado en un servidor Hetzner (www.hooksuite.de) con Docker Compose y, desde la entrega de mayo de 2026, **nadie volvió a tocarlo**: el máster encadenó otras actividades y la Práctica 3 lo retoma tal como quedó.

Qué nos encontramos (verificación en vivo, 29-sep y 4-oct)

- La herramienta **seguía en pie** desde mayo, sin cambios: DNS → Hetzner, nginx/1.31.0, uvicorn y los 36 endpoints de la API respondiendo.
- Núcleo funcional sólido: Spider, Repeater, Intruder, Utilidades (hash/encoder/regex), sesiones por token UUID y WebSocket operativos.
- Dos hallazgos que **reorientan** la Práctica 3 (detalle en el apartado 7):
- La API :8000 estaba **abierta a internet sin ninguna autenticación**, incluidos los endpoints que lanzan tráfico contra terceros.
- En el historial de git había claves de API reales (secretos expuestos).
- El módulo de IA (RF-08), que es lo que da sentido al «con IA» del producto, **nunca llegó a operar** salvo pruebas muy iniciales de la P1: panel construido pero inactivo.

Lo que la P1 nos enseñó a su costa: el saldo de la API

En las primeras pruebas del módulo de IA se cargaron 5 € de saldo en la API y se agotaron en muy poco tiempo. La causa está **reconstruida del historial de git** para esta memoria, no de lo que nadie recordara: el módulo ejecutaba una auditoría completa **en el arranque**, y el servicio está declarado con `restart: unless-stopped`, de modo que cuando el proceso terminaba Docker lo relanzaba y empezaba otra auditoría. Un bucle de auditorías mientras el contenedor estuviera levantado.

Se corrigió el 17 de mayo de 2026 con dos cambios: el módulo pasó a **esperar instrucciones** en lugar de auditar al arrancar —lo que además impide que el proceso termine, y con ello que Docker lo reinicie— y se dejó el análisis en modo simulado por defecto, como freno de mano.

Por qué figura en el punto de partida y no como anécdota: es el único incidente de la P1 con coste económico directo, y la P3 lo hereda a medias. El disparo automático está curado, pero el **tamaño** de una auditoría nunca se acotó (apartados 8 y 10). Es además una de las razones por las que el módulo de IA llegó a la P3 sin haber operado.

Numeración de requisitos

La P1 **no numeraba** los requisitos. El enunciado de la P3 (apartado 3) obliga a numerarlos ahora (RF-xx / RNF-xx) y a usar esa numeración en toda la práctica. La lista de abajo (apartado 4) está **derivada del informe de la P1** —de lo que el producto prometía y hace— y **PENDIENTE DE VALIDAR POR EL GRUPO** en la reunión antes de darla por firme.

Clasificación inicial de requisitos

Usando las cinco categorías del enunciado. El recuento es provisional, a validar por el grupo junto con la numeración.

Estado	Qué significa	Requisitos
Cumplido	Implementado en la P1 y funciona; en la P3 se verifica con prueba y evidencia	RF-01, RF-03, RF-04, RF-05, RF-06, RF-07, RF-11 · RNF-01, RNF-02, RNF-03, RNF-05, RNF-08 (12)
Parcial	Existe algo, pero incompleto o con fallos; hay que terminarlo y probarlo	RF-08 · RNF-04, RNF-07 (3)
Pendiente	Definido, pero sin implementar; se hace desde cero	RF-09, RF-10, RF-12 · RNF-09 (4)
A revisar	El feedback o la experiencia indican que estaba mal planteado; se reformula con justificación	RF-02 · RNF-06 (2)
Descartado	Se decide no implementarlo (debe ser la excepción y justificarse)	— ninguno de momento

⚠ HUECO — responsable: **José María**. Feedback docente de la P1: la calificación y los comentarios del equipo evaluador NO constan en el material de la línea. El enunciado los pide expresamente en este apartado. Conseguirlos (José María / portavoz) y resumir aquí qué se nos señaló y cómo lo aborda la P3.

4. Requisitos

Lista completa y numerada. Numeración fijada en la P3 (la P1 no la tenía) a partir del informe de la P1;

PENDIENTE DE VALIDAR POR EL GRUPO. El estado es el de partida (apartado 3); se actualiza a medida que la P3 avanza.

Requisitos funcionales (RF)

ID	Requisito	Estado P1
RF-01	Dashboard web accesible desde cualquier navegador, sin instalar nada en el cliente	Cumplido
RF-02	El servidor ejecuta las peticiones HTTP en nombre del auditor (proxy del lado servidor)	A revisar
RF-03	Spider: rastreo automático del objetivo (BFS, scope al dominio, extrae formularios), velocidad configurable	Cumplido
RF-04	Auditoría autenticada: sesión httpx persistente compartida por Spider/Repeater/Intruder que hereda las cookies del login	Cumplido
RF-05	Repeater: modificar y reenviar cualquier petición; respuesta en 4 vistas (Raw/Pretty/Preview/Headers)	Cumplido
RF-06	Intruder: fuzzing de parámetros marcados con *; 4 tipos (SQLi, Blind SQLi, XSS, genérico); resultados en tiempo real	Cumplido
RF-07	Utilidades: Encoder/Decoder (Base64/URL/HTML/JWT), Hash (MD5/SHA1/SHA256/SHA512), Regex Tester, Payload Generator	Cumplido
RF-08	Análisis y clasificación de vulnerabilidades con IA (Claude) según OWASP → panel Vulnerabilidades	Parcial
RF-09	Captura de tráfico HTTP en tiempo real (DevTools / CDP) → panel Red con código de colores	Pendiente
RF-10	Ejecución de ataques con navegador real (Playwright)	Pendiente
RF-11	Soporte multi-usuario simultáneo mediante tokens de sesión UUID	Cumplido
RF-12	Registro y login individual con JWT + aislamiento de historial/auditorías por usuario	Pendiente (mejora)

Requisitos no funcionales (RNF)

ID	Requisito	Estado P1
RNF-01	Accesible desde navegador sin instalar software en el cliente	Cumplido
RNF-02	Actualización en tiempo real vía WebSocket (pings cada 30 s, eventos {type,payload})	Cumplido
RNF-03	Desplegado en servidor Hetzner con contenedores Docker	Cumplido

RNF-04	Código en GitHub con historial de commits que refleje el trabajo del grupo	Parcial
RNF-05	Estabilidad ante carga: límite de concurrencia para no saturar producción	Cumplido
RNF-06	Operación degradada cuando la IA no responde o no supera el umbral de confianza	A revisar
RNF-07	Seguridad del propio producto: secretos fuera del repo, control de acceso, validación de entradas, dependencias sin CVEs	Parcial
RNF-08	Firewall dinámico por sesión (abre un puerto por sesión, lo cierra al logout o tras 4 h)	Cumplido (a verificar)
RNF-09	TLS/HTTPS en los endpoints públicos (Let's Encrypt)	Pendiente

Requisitos modificados o descartados (con justificación)

El enunciado pide que todo requisito modificado o descartado lleve su versión original, la nueva y el motivo.

RF-02 — proxy del lado servidor (modificado)

- **Versión original (P1):** interceptación del tráfico del navegador mediante archivo de autoconfiguración de proxy (PAC) + WebSockets.
- **Versión nueva:** el servidor ejecuta las peticiones directamente con `httpx`, sin interponerse en el navegador del auditor.
- **Motivo:** en la P1, el modelo PAC quedó expuesto y se saturó con tráfico de bots. El pivote a `httpx` directo elimina esa superficie y simplifica el flujo. Es el caso de libro de «requisito reformulado con justificación».
- **Cierre en la Fase 2 (6-oct): el PAC se retiró del código.** Hasta entonces el producto seguía sirviendo el fichero de autoconfiguración aunque el modelo estuviera abandonado desde la P1. Se comprobó que estaba muerto antes de tocarlo: su único consumidor en la interfaz era un componente **que ningún otro fichero importaba**, y el segundo (el lanzador de navegador de RF-09) no está integrado y además apunta a un puerto que la Fase 1 había cerrado, así que ya estaba roto por su cuenta. Se retiraron la ruta del backend, sus dos bloques en Nginx y el componente.
- **Ganancia de seguridad no buscada.** El PAC era la **única** ruta que tenía que quedar sin autenticar, porque un navegador no envía credenciales al pedirlo, y por eso llevaba una excepción expresa en el guardián de la Fase 2. Al jubilarlo, **toda** `/api` exige token sin excepciones: no queda ningún hueco que mantener ni que explicar. Una excepción de seguridad que se puede eliminar vale más que una excepción bien documentada.

RNF-06 — operación degradada (a revisar)

- **Original:** enunciado como principio en la P1, sin demostrar.
- **Nuevo:** la operación degradada se demuestra haciendo que un fallo de la IA sea **distinguible** de un análisis correcto. El clasificador pasa a devolver tres estados donde hoy hay dos: *vulnerable* (con confianza suficiente), *analizado y limpio*, y *no analizado*, este último acompañado del motivo. Encaja con el criterio P3 «se comporta razonablemente ante errores», y se prueba a propósito: con una clave de API inválida el panel debe decir *no analizado*, *no limpio*.
- **Corrección de este documento (8-oct).** Este apartado prometía demostrar RNF-06 «con un umbral sobre el campo `confianza`». Ese umbral **ya existía** en el código desde la P1 —`CONFIDENCE_THRESHOLD = 60`, aplicado en los dos analizadores—, así que la demostración prometida no demostraba nada: se iba a presentar como mejora de la P3 algo ya construido. Se deja escrito en vez de corregirlo en silencio, porque el error está en lo que este documento prometió, no en el producto.
- **Lo que de verdad faltaba, y no estaba en ningún pendiente.** Cuando la IA no puede analizar, el clasificador devuelve `None`, que es **exactamente** lo que devuelve cuando ha analizado y no hay nada: el panel presenta «sin vulnerabilidades» sin distinguir si eso es un resultado o una ignorancia. Y en el camino de fallo de la API es aún más brusco: el cliente lanza la excepción al agotar los reintentos y nadie la captura, de modo que **se interrumpe la fase de ataque completa** en lugar de continuar sin ese análisis. Lo uno no es operación degradada sino silencio, y lo otro no es degradación sino caída.
- **Aplicado y probado el 9-oct (commit 8ebee124).** Los tres estados ya existen en el código: el clasificador devuelve estado: `"analizado"` cuando hay hallazgo, `None` cuando ha analizado y está limpio, y estado: `"no_analizado"` con su motivo cuando no ha podido analizar. Hizo falta añadir además

el módulo `ia/esquemas.py`, que no existía: el cliente reescrito exige un esquema de respuesta y ninguna de las cuatro plantillas de consigna definía uno, de modo que el plan tal como estaba escrito habría fallado en la primera llamada. La prueba automática `ia/tests/test_tres_estados.py` recorre los tres estados en las cuatro vías de análisis con un cliente simulado —sin clave de API ni red, coste cero— y pasa *8 de 8*; sobre el código anterior daba *0 de 8*, y el error era precisamente el descrito arriba: la fase de análisis se caía entera. **Lo que aún impide que se vea en el panel** es un filtro del orquestador que compara la confianza contra 0.6 cuando la escala es de 0 a 100: descartaría el estado degradado antes de mostrarlo. Es un defecto distinto, ya documentado, y se corrige por separado.

RF-12 — registro y login individual con JWT (mejora, no requisito P1)

- No era requisito de la P1 (venía del road map de mejoras). En la P3 cuenta como **mejora**: suma, pero no compensa un requisito original sin cumplir. Se aborda por su valor de seguridad: es lo que cierra la fuga de datos entre usuarios documentada en los apartados 7 y 8.
- **Corrección de este documento (6-oct)**. Este apartado se titulaba «login individual con JWT», sin la palabra **registro**. El requisito, tal como está escrito, pide las dos cosas: que cada persona **se cree su cuenta** y que entre con ella. Al redactar se estrechó el requisito a la mitad sin declararlo; se deja constancia porque un requisito recortado en silencio es peor que un requisito incumplido y explicado.
- **Cómo se resuelve el registro, y por qué no es abierto**. La persona se da de alta ella misma y elige su propia contraseña —nadie más la conoce, ni quien administra, que es el punto del requisito—, pero necesita un **código de invitación** que reparte el grupo. La razón es el propio producto: HookSuite lanza tráfico contra terceros, así que con altas anónimas cualquiera podría usar el Spider y el Intruder contra quien quisiera desde nuestra infraestructura. El código es obligatorio por configuración: si faltara, el servicio no arranca, en lugar de arrancar con el registro abierto sin que nadie lo note.
- **Lo que obliga a persistir, y lo que no**. Las **cuentas** se guardan en un volumen, porque el despliegue reconstruye los contenedores y unas cuentas que se evaporasen en cada despliegue harían inútil el registro. El **estado de sesión** sigue siendo volátil a propósito, por las razones del apartado 10: son cosas distintas y conviene no confundirlas.

> **Descartados**: ninguno por ahora. El enunciado pide que sean la excepción; si alguno se descarta (candidato: RF-09/DevTools si aprieta el tiempo) se justificará aquí por escrito.

5. Arquitectura y decisiones técnicas

HookSuite se despliega como un conjunto de contenedores Docker coordinados por `docker-compose`, todos en una red interna `hooksuite-net`, con Nginx como única puerta de entrada. Arquitectura verificada fichero a fichero en la cocina local el 4-oct (`docker-compose.yml` + `infra/nginx.conf`).

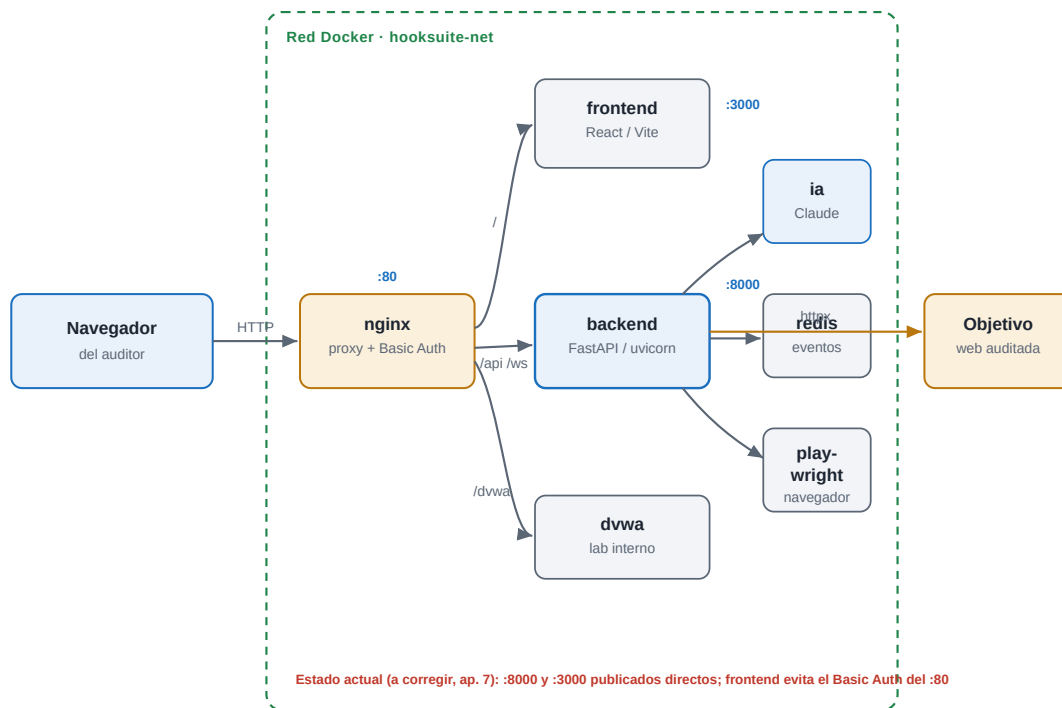


Figura 1: Arquitectura de contenedores de HookSuite: Nginx es la única entrada; el backend FastAPI orquesta Spider/Repeater/Intruder/IA/Playwright y sale al objetivo auditado con `htpx`. Desde la Fase 2 la autenticación ya no está en Nginx sino en el backend, que exige token en toda `/api` y `/ws/` (apartado 7).

Componentes

Componente	Tecnología	Rol	Puerto (prod.)
nginx	nginx:alpine	Proxy inverso y único punto de entrada; enruta a frontend y backend (el laboratorio salió del proxy en la Fase 1 y el control de acceso pasó al backend en la Fase 2)	80-80
frontend	React + Vite (build propio)	Dashboard web (SPA) servido como estático	3000-80
backend	FastAPI + uvicorn (build propio)	API de auditoría: proxy <code>htpx</code> , Spider, Repeater, Intruder, utilidades, vulnerabilidades y WebSocket	8000-8000
redis	redis:7-alpine	Bus de eventos y estado efímero entre servicios	interno
ia	Python + SDK Anthropic (build propio)	Clasificador de vulnerabilidades con Claude (RF-08)	interno
playwright	Python + Playwright (build propio)	Navegador real para ataques dirigidos (RF-10)	interno
dvwa	vulnerables/web-dvwa	Laboratorio de pruebas, solo accesible por la red interna	interno

Enrutado (Nginx)

Toda petición entra por Nginx, que reparte por ruta (`infra/nginx.conf`):

- `/` → **frontend**: sirve el panel de acceso propio del producto. Hasta la Fase 2 iba protegido con Basic Auth; ya no.
- `/api/` y `/ws/` → **backend** (API y WebSocket). Desde la Fase 2, **ambas exigen token**; sin él responden 401.
- `/check/` y `/proxy.pac` → **retiradas en la Fase 2**, al jubilar el fichero de autoconfiguración de proxy (apartado 4, RF-02).

Desde la Fase 1, Nginx es además la **única** entrada: backend y frontend dejaron de publicar puerto propio, y la ruta hacia el laboratorio vulnerable se retiró del proxy (apartado 7).

El reparto de responsabilidades cambió con la Fase 2, y conviene decirlo porque es una decisión de arquitectura, no un detalle: antes Nginx era quien autenticaba (una contraseña única, igual para todos,

que no distinguía personas) y el backend no pedía nada. Ahora Nginx solo reparte, y **quien autentica y autoriza es el backend**, que es el único que puede saber de quién es cada dato.

Decisiones técnicas

Proxy del lado servidor: pivote PAC → httpx (RF-02)

La decisión arquitectónica de más peso heredada de la P1. En origen, HookSuite interceptaba el tráfico del navegador con un archivo de autoconfiguración (PAC) + WebSockets; ese modelo quedó expuesto y se saturó con tráfico de bots. Se pivotó a que **el servidor ejecute las peticiones directamente con httpx**: menos superficie y un flujo más simple, a cambio de perder la captura pasiva del tráfico del navegador —justo lo que retoman después DevTools (RF-09) y Playwright (RF-10)—. Versión original/nueva/justificación detalladas en el apartado 4.

Frontend: reescritura a Tailwind (cambio respecto a la P1)

La interfaz que corría en producción (www.hooksuite.de) se había reescrito al framework de estilos **Tailwind**, abandonando el sistema de variables CSS propio de la P1. Esa versión —la desplegada y en uso— es la que se toma como **base del frontend en la P3**. En la Fase 0 se reconcilió al repositorio (vivía solo en la caja, sin commitear); el detalle del proceso está en el apartado 8.

Puntos a corregir, detectados al montar la cocina

- **CORS abierto a cualquier origen** en el backend (FastAPI) → **corregido en la Fase 1**: lista cerrada de orígenes, configurable por entorno (apartado 7).
- **Verificación SSL desactivada** en el cliente httpx: justificable en una herramienta de auditoría (objetivos con certificados autofirmados), pero se documenta expresamente como decisión, no como descuido.
- **Acoplamiento Nginx ↔ DVWA**: `infra/nginx.conf` hacía del laboratorio un requisito de arranque del proxy (host not found in upstream) → **resuelto en la Fase 1** al retirar su ruta del proxy: el laboratorio queda solo en la red interna y Nginx ya no depende de él.
- **Playwright fuera de la red**: en el `docker-compose.yml` el servicio playwright no declaraba `networks: hooksuite-net`, así que quedaba aislado y no hablaba con el backend ni con el laboratorio, pese a tener configurados `BACKEND_URL=http://backend:8000` y `DVWA_URL=http://dvwa:80` (causa raíz de RF-10/RF-08 sin operar) → **corregido el 9-oct** con una línea en el compose, verificado con `docker compose config`, que ahora sitúa los siete servicios en `hooksuite-net`. **Corrección declarada (R9)**: hasta ese arreglo la frase «todos en una red interna hooksuite-net» del apartado 5 era inexacta y el documento se contradecía a sí mismo; se deja dicho aquí en vez de reescribirlo en silencio.

> **Nota de reproducibilidad (cocina)**. En el mediaserver los puertos 80/3000/8000 ya los ocupan otros servicios, así que la cocina los remapea a 8880/8830/8800 mediante un override local que no toca el compose versionado. Es solo del entorno de pruebas; la arquitectura de producción es la de la tabla.

 **HUECO** — responsable: **José María + Claude**. Añadir, si procede, un diagrama de secuencia del ciclo IA ↔ Playwright ↔ Backend cuando RF-08/RF-10 estén operativos.

6. Funcionalidades implementadas

Cada funcionalidad con capturas: qué hace, cómo se usa y qué requisitos cubre.

- Proxy interceptor, Repeater, Intruder, Utilidades (hash/encoder/regex), Vulnerabilidades, Red/DevTools, módulo IA.

Defectos corregidos en la Fase 1

Diez defectos del prototipo, todos reproducidos antes de corregirlos y verificados después en producción. El detalle de cada uno (causa raíz, prueba y autoría) está en el diario del repositorio.

Qué fallaba	Causa	Requisito
El Spider entraba en bucle y martilleaba el objetivo	No canonicalizaba las direcciones: el mismo recurso generaba variantes infinitas	RF-03

El Spider podía dirigirse a servicios internos del propio servidor	Faltaba filtrar destinos no legítimos	RNF-07
El botón de detener el Spider no existía en la interfaz	El servidor aceptaba la orden, pero la pantalla no ofrecía forma de darla	RF-03
Detener el Spider no detenía nada	El bucle de rastreo no consultaba la señal de parada	RF-03
Un mismo formulario aparecía decenas de veces	No se agrupaban: se emitía uno por página visitada	RF-03
Cancelar el Intruder devolvía un error del servidor	La operación invocaba un método inexistente	RF-06
Dos operaciones del proxy devolvían error del servidor	Un error de sintaxis construía un tipo de dato equivocado	RF-02
Importar un comando <code>curl</code> rompía las direcciones seguras	Una expresión regular mal escapada recortaba el esquema	RF-05
Los avisos de error no decían qué había fallado	Se mostraba el texto crudo de la librería de red	RF-05
Dos pantallas no recibían avisos en tiempo real	El servidor guardaba una sola conexión por sesión y la segunda desplazaba a la primera	RNF-04
El agente del cortafuegos era manipulable por cualquiera	Su canal se creaba con permisos para todo el sistema y no validaba lo recibido	RNF-08
El estado de sesión podía crecer sin límite	El recolector existía pero nunca se ejecutaba, y cualquier identificador nuevo creaba una sesión	RNF-07

Sistema de acceso por usuario (Fase 2)

Funcionalidad nueva, no un defecto corregido: cada persona se crea su cuenta, entra con ella y **solo ve sus propias auditorías**. Cubre RF-12 y es lo que cierra las cuatro fugas del apartado 7.

- **Registro con código de invitación.** La persona elige su contraseña y nadie más la conoce —ni quien administra—, pero necesita un código que reparte el grupo. No es abierto a propósito: HookSuite lanza tráfico contra terceros, así que con altas anónimas cualquiera podría usar el Spider o el Intruder contra quien quisiera desde nuestra infraestructura. El código es obligatorio por configuración: si falta, **el servicio no arranca**, en vez de arrancar con el registro abierto sin que nadie lo note.
- **Entrada con token firmado por el servidor.** Las contraseñas se guardan como hash, nunca en claro. El algoritmo de firma está fijado en el servidor y no se lee del propio token, porque aceptarlo del token permite presentar uno sin firma y que el servidor lo dé por bueno.
- **Cada usuario, su espacio.** El historial, las cookies capturadas y los hallazgos cuelgan de la cuenta de quien los generó. El identificador de ese espacio **sale siempre del token**: ni de la URL ni del cuerpo de la petición, para que el cliente no pueda elegir dónde escribe.
- **Cuentas persistentes.** Se guardan en un volumen propio, porque el despliegue reconstruye los contenedores y unas cuentas que se evaporasen en cada despliegue harían inútil el registro. El estado de sesión, en cambio, sigue siendo volátil a propósito (apartado 10): son cosas distintas.

Nota sobre el último: no era el defecto que teníamos anotado. El pendiente decía «las sesiones no se conservan al reiniciar»; al abrirlo resultó que el problema real era el contrario y más grave —podían crecer sin límite desde fuera—. La volatilidad se justifica en el apartado 10; lo que se corrigió fue el crecimiento.

⚠ **HUECO** — responsable: **José María + Claude**. Documentar cada funcionalidad (qué hace, cómo se usa, qué requisitos cubre) con capturas de producto; las de pantalla las saca José María (R6), tras la Fase 3, cuando el login nuevo cambie las pantallas. El commit de cada módulo se firma a nombre de su responsable cuando toque (R2): Ivan/frontend, Macarena/backend, Nacho/playwright, Carlos/DevTools, José María/IA.

7. Seguridad del producto

Modelo de amenazas, secretos, autenticación, validación de entradas, dependencias y datos personales. (Base en REQUISITOS.md §3.) Contenido ya verificado en esta práctica:

Secretos: el incidente de las claves

En el historial de git había claves de API de Anthropic. Verificado el 4-oct: **las cuatro están muertas** (la API responde 401 / "API key is invalid."). Aun así, el enunciado (§7) obliga a limpiarlas del historial, no basta con borrarlas en un commit nuevo.

- El 4-oct se **reescribió el historial** con `git-filter-repo`: se redactaron las claves por patrón y se eliminaron los `.env` rastreados y la basura (`node_modules`, `venv` de Windows), conservando el código, las 7 ramas y el `.env.example` (con la clave ya redactada).
- **Ejecutado en GitHub el 4-oct** (force-push de las 7 ramas, con copia de seguridad previa). Verificado sobre un clon nuevo: **0 claves en todo el historial** de las ramas. El repositorio público queda sin secretos.
- Salvedad: GitHub conserva un tiempo los refs internos de los **pull requests** antiguos apuntando a los commits viejos; los purga por su cuenta. Como las claves están muertas, no supone riesgo.

LIMPIEZA EJECUTADA Y VERIFICADA

Exposición de la caja: cómo estaba

Estado de partida, verificado en vivo el 29-sep y el 4-oct (evidencia fotográfica en el apartado 8):

- La API `:8000` **no tenía autenticación**: lo declaraba su propia `openapi.json` (40 operaciones sin seguridad) y `/health` respondía sin credenciales.
- El Basic Auth del `:80` se **evitaba** sirviendo el frontend directamente por `:3000`.
- El servidor **no tenía cortafuegos** (política de aceptación por defecto) ni protección contra fuerza bruta, y acumulaba unos **127.000 intentos de acceso en 7 días**.
- DVWA **no** estaba expuesto a internet (solo red interna) — corrige un supuesto previo nuestro.

Endurecimiento aplicado en la Fase 1 (5-oct)

Cierre de la exposición

La API y el frontend dejaron de publicar puerto propio: **la única entrada es el proxy**. No bastaba con retirarlos del `docker-compose.yml`, porque el frontend llevaba la dirección del servidor incrustada en tiempo de compilación y hablaba directamente con la API; hubo que **reconstruirlo para que use el mismo origen** y que sus llamadas (incluido el WebSocket) pasen por el proxy. Comprobado desde fuera tras el despliegue: los puertos directos **rechazan la conexión**, el proxy responde, y el paquete del frontend ya no contiene ninguna referencia a la dirección antigua.

Orígenes permitidos (CORS)

El backend aceptaba **cualquier** origen con credenciales, combinación además inválida por especificación: en la práctica, el servidor **reflejaba el origen de quien preguntara**, de modo que cualquier web podía hacerle peticiones autenticadas. Se sustituyó por una **lista cerrada** configurable por entorno. Comprobado: un origen arbitrario ya no recibe permiso; uno legítimo sí.

El laboratorio vulnerable, solo interno

Se retiró del proxy la ruta que lo publicaba. Queda accesible únicamente desde la red interna, para las pruebas del propio producto.

Endurecimiento del servidor

- **Cortafuegos** activo, con política de denegación por defecto y únicamente el acceso remoto y el proxy permitidos.
- **Bloqueo automático de fuerza bruta** contra el acceso remoto: en su primer minuto ya registraba 148 intentos fallidos y había bloqueado dos direcciones.
- **Acceso remoto solo por clave**: se desactivó la autenticación por contraseña. La prueba más limpia es el propio mensaje del servidor, que pasó de ofrecer «clave o contraseña» a admitir **solo clave**.
- **Limpieza de reglas abandonadas**: siete reglas heredadas de la P1 —una de ellas con la dirección pública de un integrante y otra con un valor de ejemplo— se retiraron tras comprobar que ningún servicio escuchaba en esos puertos.

- **Agente de firewall duplicado:** se encontró un segundo proceso del agente, lanzado a mano en mayo y fuera del gestor de servicios, con permisos de administración del cortafuegos. Se terminó; queda uno solo, gestionado por el sistema.
- **Sistema operativo:** se estrenó el núcleo ya instalado pero sin aplicar desde mayo. Antes de reiniciar se corrigió una condición de carrera en el arranque que habría dejado el agente del cortafuegos en bucle (ver apartado 8).

Dos matices honestos, para no atribuirnos de más

- **El acceso de administrador por contraseña ya estaba bloqueado** por la configuración por defecto del sistema: los 127.000 intentos registrados nunca tuvieron ninguna posibilidad. Lo que se cerró fue la autenticación por contraseña **a nivel general**. Se hizo igual porque deja de depender de un valor por defecto que una actualización podría cambiar, y porque cierra la puerta a cualquier usuario futuro.
- **El cortafuegos del sistema no filtra los puertos que publica Docker**, cuyas reglas se evalúan antes. Quien cerró de verdad la API y el frontend fue el despliegue, al dejar de publicarlos. El cortafuegos protege el resto del servidor, no esos puertos.

EXPOSICIÓN CERRADA Y SERVIDOR ENDURECIDO

Autenticación y aislamiento: de cuatro fugas a cero (Fase 2)

Hasta la Fase 2, la API y el WebSocket **no tenían autenticación** por detrás del proxy: el único control era el Basic Auth de la entrada, que además no distinguía personas. Eso no era una carencia, sino **cuatro fallos distintos**, los cuatro medidos en la cocina el 6-oct antes de tocar una línea de código. La tabla enfrenta cada uno con lo que lo cierra; la evidencia de los dos lados está en el apartado 8.

Cómo estaba (medido el 6-oct)	Cómo queda (Fase 2)
La API entera respondía sin credencial. La asimetría lo delataba: GET /health devolvía 401 por el Basic Auth de la entrada, pero GET /api/session/new devolvía 200 y entregaba un token. El proxy protegía unas rutas y dejaba /api abierta.	Un guardián aplicado por router , no ruta por ruta, para que una ruta nueva nazca protegida. Toda /api y /ws/ exigen token: sin él, 401 con cabecera WWW-Authenticate. El Basic Auth de la entrada se retiró, porque la credencial la pide ahora quien debe.
El token de sesión no era una credencial, era un nombre. El backend creaba sesión para cualquier cadena inventada, así que no había frontera que violar: bastaba escribir un token cualquiera en la URL.	El token lo firma el servidor y el espacio de datos se deduce del token, nunca de la URL ni del cuerpo. El navegador dejó de inventarse su identificador. La ruta que repartía identificadores sin valor de credencial se retiró.
La cookie de sesión de la web auditada se compartía entre todos. Se guardaba en una tabla global indexada solo por dominio, sin dueño; un segundo usuario leía la cookie capturada por el primero, y se obtenía además sin token.	Las cookies viven dentro de la sesión de cada usuario y la tabla global desapareció. Verificado: el segundo usuario ya no obtiene la cookie del primero.
Los hallazgos se difundían a todos los paneles. Una vulnerabilidad publicada sin token aparecía en la sesión de dos auditores distintos: en una herramienta de auditoría, eso es una fuga de datos del cliente de otro.	Los dos endpoints que admitían peticiones sin token escriben ahora solo en el espacio de quien llama. Verificado: el segundo usuario no ve el hallazgo del primero.

Un quinto fallo, que introdujo la propia corrección

Se documenta porque el enunciado pide las pruebas que fallaron, y porque la lección vale más que el acierto. El guardián comprobaba que el espacio de datos nombrado **en la ruta** fuera el tuyo, pero **cinco rutas reciben ese nombre en el cuerpo de la petición**, y ahí no miraba nadie: un usuario autenticado podía escribir en el espacio de otro con solo ponerlo en el cuerpo. Es el mismo error que este documento le reprocha al código anterior —dejar que el cliente elija dónde escribe— un nivel más abajo.

La peor de las cinco fue el Repeater, y se escapó del primer inventario por buscarla con el nombre de modelo equivocado. Ahí el daño no era ensuciar el historial ajeno: el Repeater mantiene **un cliente HTTP persistente por usuario** que acumula las cookies del objetivo auditado —el mecanismo que da servicio a RF-04—, de modo que nombrar a otro en el cuerpo reutilizaba **su sesión ya autenticada contra la web auditada**.

Se descartó la corrección evidente, que era validar ese campo contra el token: funciona, pero deja el dato en manos del cliente y obliga a acordarse en cada modelo nuevo. Las rutas toman el espacio del token y

descartan lo que venga en el cuerpo: lo que no se lee no se puede falsear. Comprobado como ataque real (apartado 8).

Desplegado y verificado en producción (8-oct)

Hasta esta fecha, este apartado decía expresamente que no podía afirmar nada de la instalación pública: todo estaba **verificado en la cocina**, y la caja seguía sirviendo la Fase 1 con las cuatro fugas abiertas. Se dejó escrito así a propósito, porque la tentación de contar una mejora en presente antes de que llegue a producción es justo como una memoria acaba describiendo un producto que no existe. El 7 de octubre se comprobó además **en vivo y en la propia instalación pública** que las fugas no eran teóricas: la API respondía sin credencial a cualquiera.

El despliegue se hizo el **8 de octubre a las 06:52 UTC**, en cuatro pasos con punto de retorno previo: se etiquetaron las imágenes en servicio y se copió la configuración, se mergeó el cambio a la rama principal, se trajo el código a la caja con `git reset --hard` y se generaron los secretos, y se reconstruyeron los contenedores. La verificación inmediata, con el sistema ya en marcha:

Qué se midió	Resultado
Entrada del panel (/)	200 — carga sin pedir Basic Auth : la contraseña compartida está retirada
API (/api/auth/yo)	401 — exige token ; la fuga nº 1 de la columna izquierda, cerrada
Reinicios del backend	0 — prueba que los secretos llegaron: el código se niega a arrancar sin ellos
Contenedores recreados	backend, frontend y nginx; los otros cuatro, intactos a propósito

El dato de los **cero reinicios** no es decorativo: el acceso por usuario se diseñó para **no arrancar** si faltan el secreto de firma o el código de invitación, en lugar de arrancar sin autenticación pareciendo correcto. Que el contenedor esté sirviendo sin un solo reinicio es la prueba de que esa comprobación se superó.

El flujo completo del acceso, capturado en la instalación pública el mismo día del despliegue:

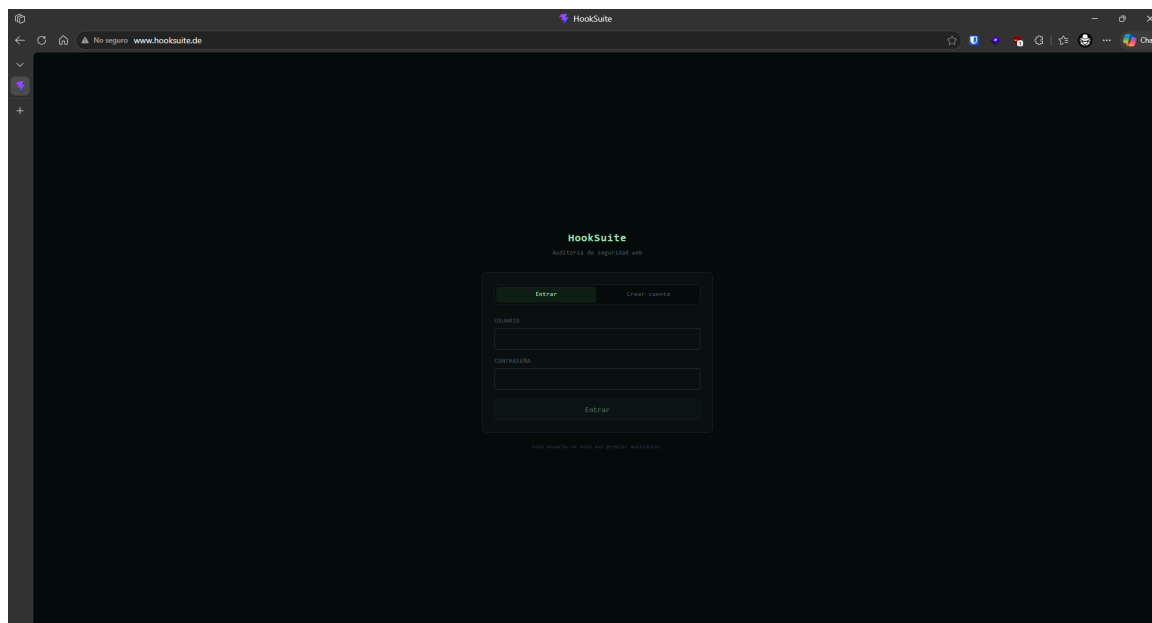


Figura 2: RF-12 — la entrada de `www.hooksuite.de` carga **sin** el cuadro de Basic Auth que antes pedía una contraseña compartida por todo el grupo. La puerta la guarda ahora el acceso por usuario.

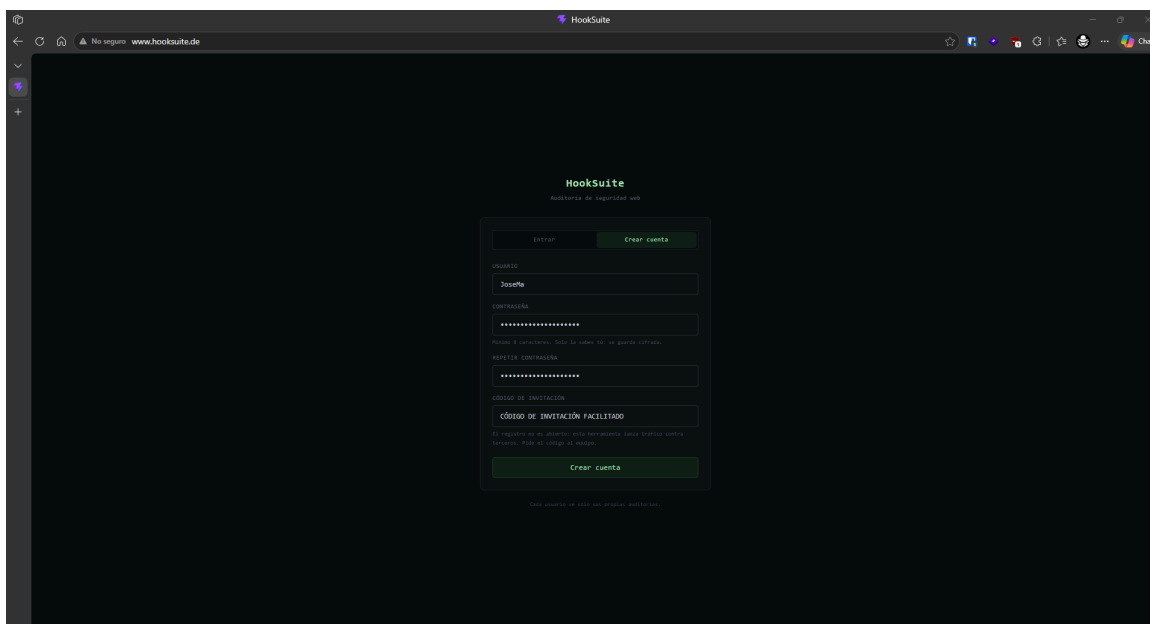


Figura 3: RF-12 — el alta exige un **código de invitación**. El registro no es abierto a propósito: la herramienta lanza tráfico contra terceros, y con altas anónimas cualquiera atacaría a quien quisiera desde nuestra infraestructura. El campo de la imagen muestra el texto de ayuda, no el código.

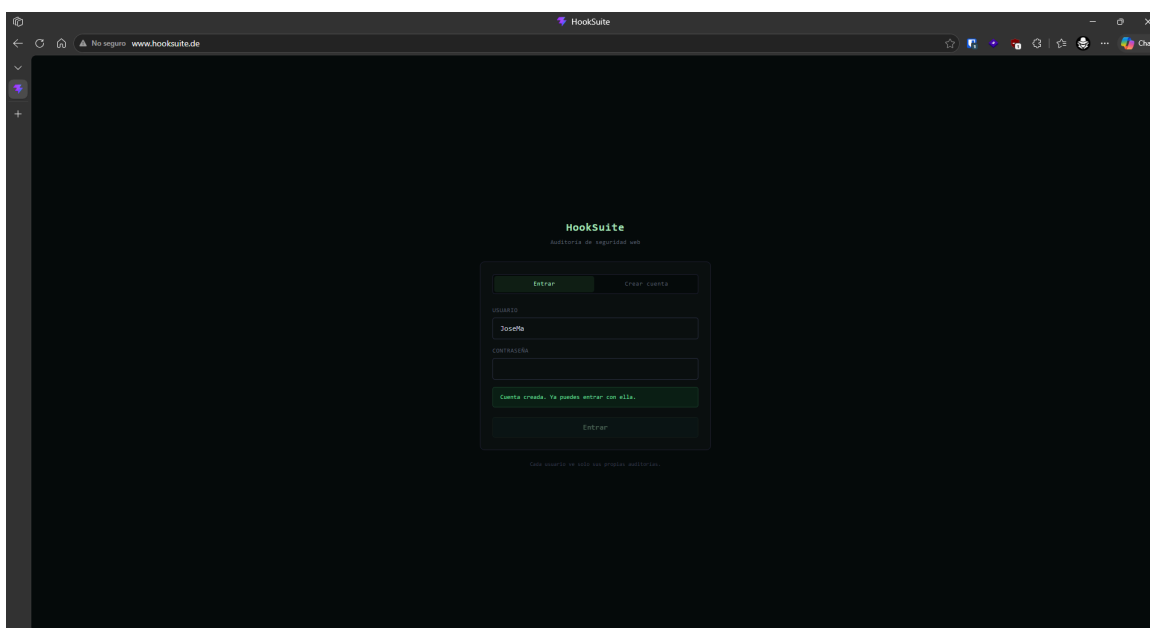


Figura 4: RF-12 — confirmación del alta del primer usuario de producción, creado con el código de invitación. La instalación arrancó deliberadamente con cero usuarios.

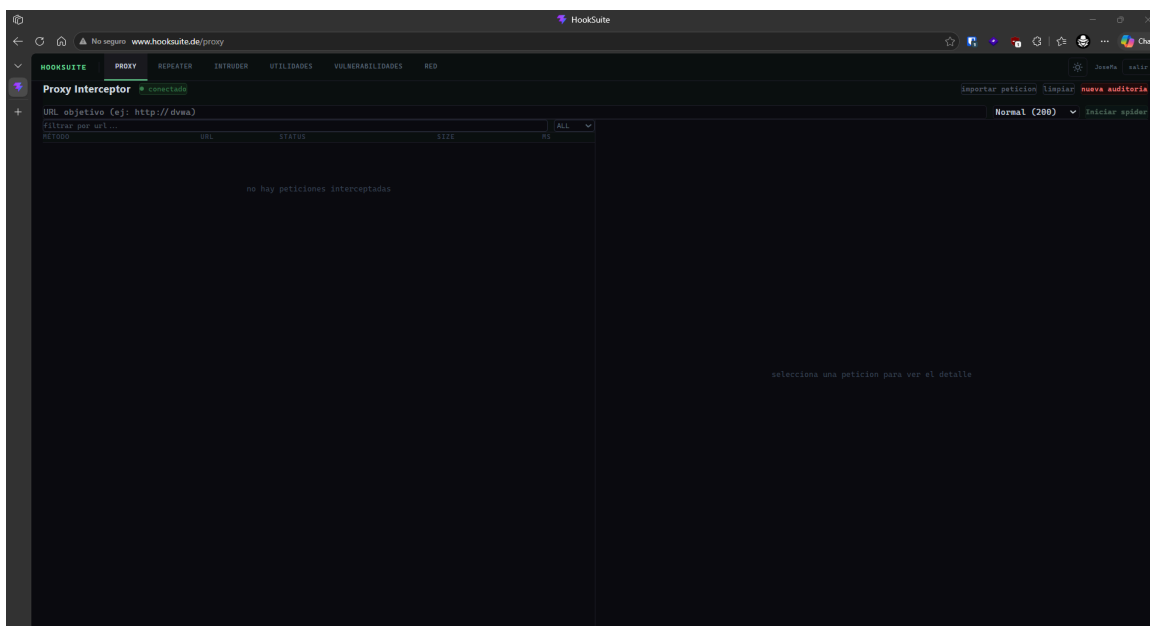


Figura 5: RF-12 y RNF-07 — sesión iniciada: el usuario figura en la cabecera y el interceptor aparece como **conectado**, lo que prueba que el WebSocket viaja también autenticado y no solo la API.

Lo que sigue sin poder afirmarse, y no se disfraza. El dominio publica también una dirección IPv6, y por ese camino la comprobación devolvió 404 en lugar del panel. No está confirmado qué ve un visitante real que llegue por IPv6, porque desde el equipo de pruebas no hay salida por esa vía. Mientras no se compruebe desde una red con IPv6, la disponibilidad solo está demostrada por IPv4.

CUATRO FUGAS CERRADAS Y VERIFICADAS EN PRODUCCIÓN (8-oct, 06:52 UTC) · PENDIENTE: COMPROBAR LA LLEGADA POR IPv6

Lo que sigue sin cubrir

- **Sin TLS (RNF-09):** el acceso viaja en claro, así que el token de sesión es interceptable por quien esté en el camino. Pendiente de configurar en el proxy.
- **Credencial de servicio para los módulos internos (Fase 3):** el clasificador de IA y el agente de Playwright publican sus hallazgos en rutas que ahora exigen token, y no tienen credencial. Comprobado en la instalación de producción el 7-oct: los dos contenedores **sí están en marcha**, pero ninguno llega hoy a esas rutas — el clasificador está conectado al servidor y a la espera de instrucciones, que no llegan porque RF-08 todavía no tiene disparador, y el agente de Playwright no alcanza siquiera el servidor por el fallo de red descrito en el apartado 5. Es decir: el acceso por usuario no degrada nada que hoy funcione, pero en cuanto RF-08 tenga su disparador el clasificador recibirá 401 si no se le da credencial de servicio.

> **Corrección declarada de este documento (R9).** Una versión anterior de este apartado justificaba lo anterior diciendo que ninguno de los dos módulos estaba en marcha. Era falso: valía para el entorno de pruebas, donde no se levantan, y se escribió sin comprobar la instalación de producción. La conclusión se sostiene, pero por un motivo distinto del que se dio, y la diferencia importa: uno de los dos está vivo y conectado al servidor.

- **Modelo STRIDE, validación de entradas, dependencias y datos personales**, abajo.

⚠ HUECO — responsable: **José María**. Completar el modelo STRIDE, validación de entradas, dependencias y tratamiento de datos personales.

8. Pruebas y evidencias

Plan de pruebas, casos ejecutados, resultados y las pruebas que fallaron con su explicación. El detalle caso a caso (PR-01...PR-20) se mantiene en la matriz del apartado 9 y en el diario del repo.

Evidencia de exposición (RNF-07 / PR-16), capturada el 4-oct

La prueba PR-16 —llamar a la API sin credenciales— y la verificación del frontend confirman que, antes de endurecer, el producto quedaba accesible sin autenticación. Capturas obtenidas antes de cualquier arreglo (R6: la evidencia de un fallo se captura antes de corregirlo, porque el propio arreglo la destruye):

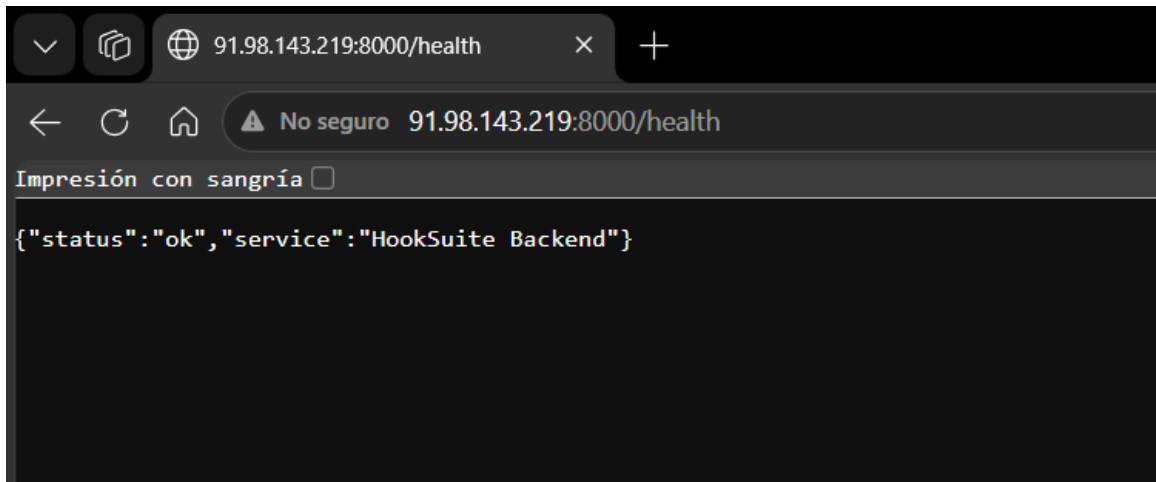


Figura 6: RNF-07 / PR-16 — GET /health de la API responde 200 sin credenciales ni cabecera `WWW-Authenticate`: la API no exige autenticación.

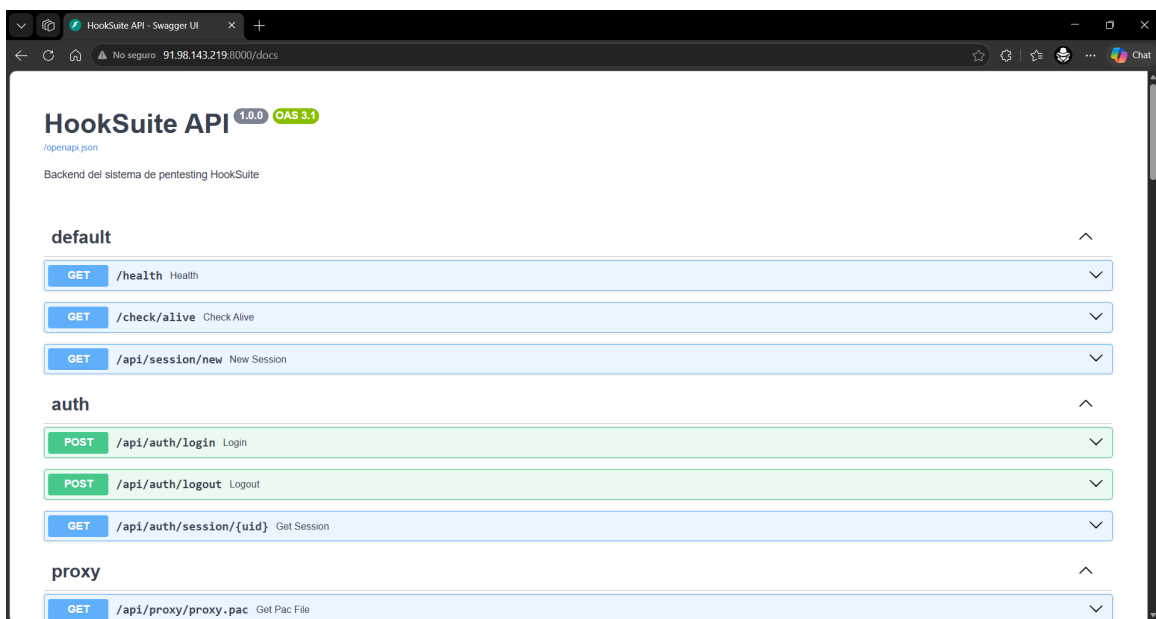


Figura 7: RNF-07 / PR-16 — la documentación Swagger (/docs) queda abierta a cualquiera, exponiendo las 40 operaciones de la API, incluidas las que lanzan tráfico contra terceros.

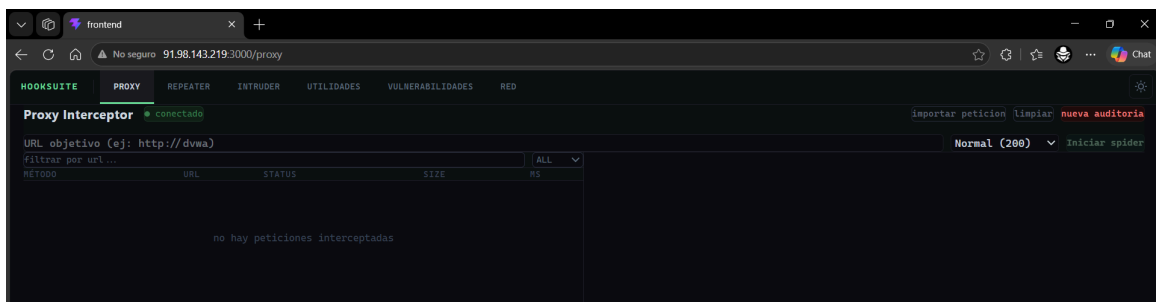


Figura 8: RNF-07 — el frontend servido directamente por :3000 carga 200 sin Basic Auth, evitando por completo el login de Nginx del :80.

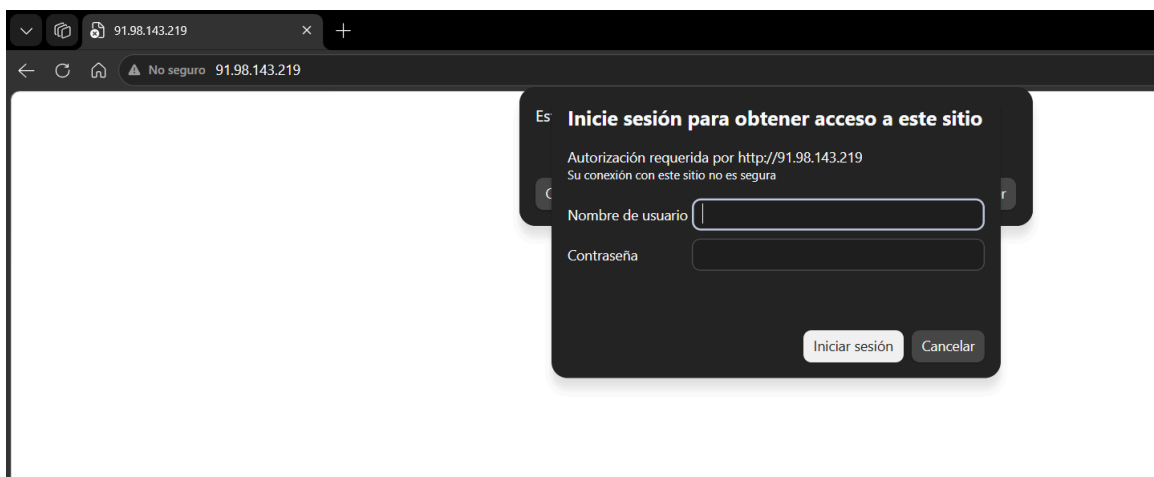


Figura 9: RNF-07 — por contraste, la entrada «correcta» (:80 tras Nginx) sí pide Basic Auth: el control existe, pero se puede rodear por el puerto directo.



Figura 10: RNF-07 — la credencial documentada en el informe de la P1 es rechazada con 401 por nginx/1.31.0: la credencial publicada ya no sirve, lo que refuerza el cambio de modelo de acceso en la Fase 2.

Defectos corregidos: antes y después (RF-03)

Dos pares de capturas tomadas sobre el producto desplegado, antes y después de corregir. El objetivo de las pruebas es una web propia del equipo, autorizada para ello.

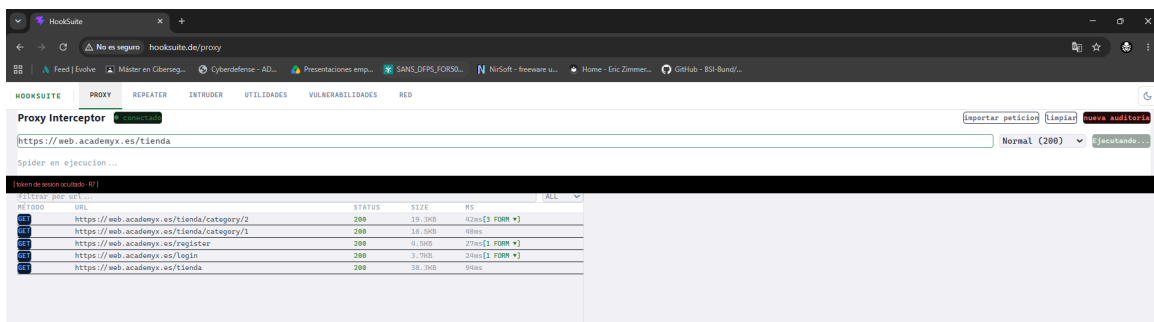


Figura 11: RF-03 — ANTES: el Spider está rastreando («Spider en ejecución...») y el único botón del panel aparece bloqueado como «Ejecutando...». No existe ninguna forma de detener el rastreo desde la interfaz.

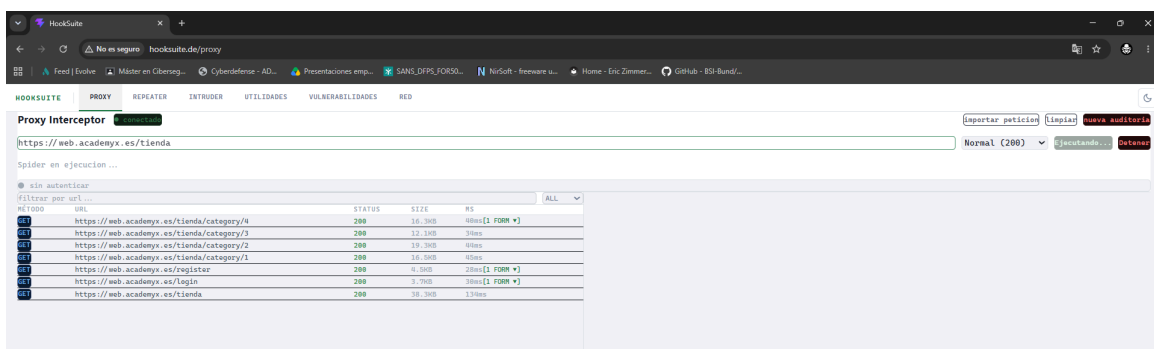


Figura 12: RF-03 — DESPUÉS: en la misma situación aparece el botón «Detener» junto al de ejecución. Obsérvese además el contador «1 FORM» en cada petición, frente a las decenas de la captura siguiente.

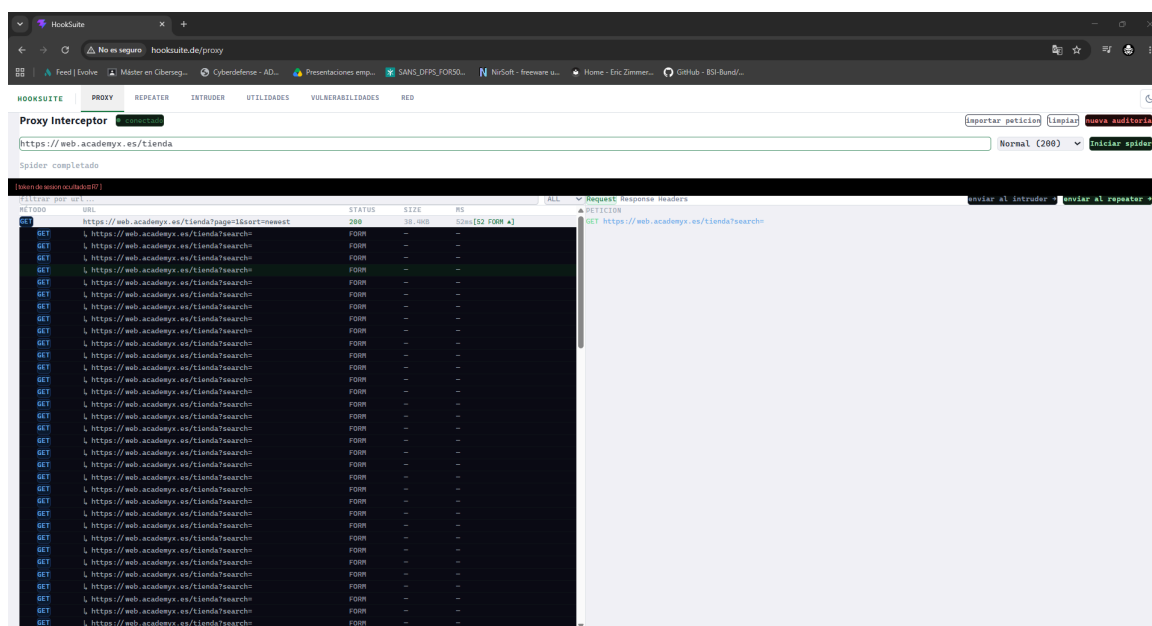


Figura 13: RF-03 — ANTES: una sola petición (/tienda?page=1&sort=newest) arrastra 52 entradas de formulario idénticas, que inundan el panel de resultados.

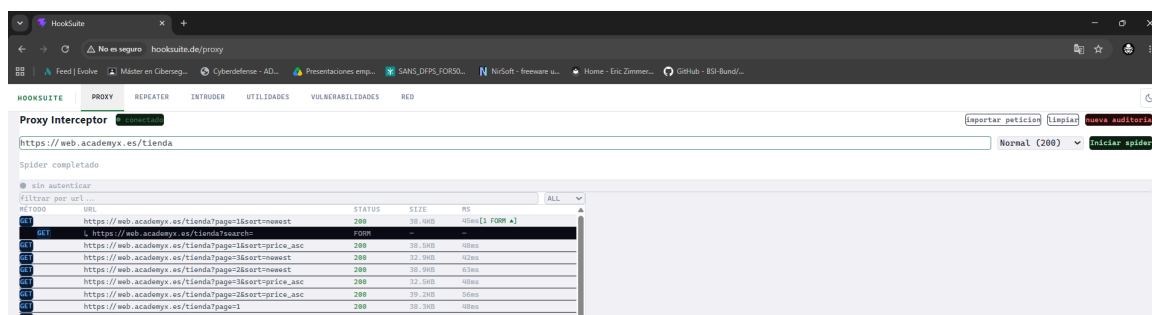


Figura 14: RF-03 — DESPUÉS: la misma petición de la misma página queda en una sola entrada. Solo cambia el contador: 52 → 1.

> **Nota de tratamiento de las capturas (R7).** Las dos del estado anterior mostraban un identificador de sesión de la web de pruebas en la franja superior; se publican con esa franja tapada, y los originales no salen del almacén interno. Las del estado posterior no lo contienen porque la sesión no estaba autenticada.

Condición de carrera detectada antes de reiniciar el servidor

El agente del cortafuegos y el servicio de contenedores no tenían orden de arranque entre sí. Si el segundo ganaba la carrera, **creaba una carpeta donde debía ir el canal de comunicación del agente** —comportamiento normal al montar un fichero que no existe— y el agente quedaba en un bucle de reinicios. El servidor llevaba sin reiniciarse desde mayo, así que nunca se había puesto a prueba. Se corrigió fijando el orden de arranque antes de reiniciar; tras el reinicio, el contador de reinicios del agente marcaba **cero**: la carrera no llegó a producirse.

Otras evidencias ya en mano

- **Incidente de las claves y limpieza del historial:** ensayo de reescritura verificado antes/después (3 → 0 claves), detallado en el apartado 7.
- **Salidas de terminal** (health sin login, openapi.json sin seguridad, cierre de puertos, mensajes de error) guardadas como texto en el repo, no como foto (R7: así ninguna captura publica por error un secreto).

Evidencia de las fugas de aislamiento (RF-12), capturada el 6-oct

Cuatro pruebas lanzadas contra la cocina a través del proxy, como las haría un cliente, **antes de tocar una sola línea de código de la Fase 2**: el propio arreglo borra esta evidencia, y el apartado pide las pruebas que fallaron con su explicación. Los valores de cookie empleados son inventados a propósito, de modo que la captura no contiene ningún identificador real (R7).

Las cuatro confirmaron el fallo: la API responde 200 sin credencial mientras /health responde 401; un token inventado obtiene sesión propia; un segundo usuario lee la cookie de sesión capturada por el primero —y también se obtiene sin token—; y una vulnerabilidad publicada sin token aparece en la sesión de dos auditores distintos.

La captura íntegra, con la causa en fichero y línea de cada una, está en evidencias/fugas-aislamiento-06oct.md. Una corrección respecto a lo que teníamos anotado: la difusión no ocurre solo en las dos llamadas que emiten a todas las sesiones, sino también en los dos endpoints que admiten peticiones sin token, que además **persisten** el dato en la sesión de todos los usuarios. Son cuatro puntos de difusión, no dos.

El aislamiento, verificado después de cerrarlo (RF-12)

Con el sistema de acceso puesto, se repitieron las mismas pruebas contra la cocina. **Once comprobaciones, once en verde**: alta de dos usuarios distintos; entrada de ambos; contraseña incorrecta rechazada con 401 y no con un error del servidor; la identidad del portador del token resuelta correctamente; la API respondiendo con token; 403 al intentar tocar el espacio de otro con token propio —es decir, autenticación y autorización, que son cosas distintas—; el segundo usuario sin ver el hallazgo del primero; y el segundo usuario sin ver la cookie de sesión del primero, que era la tercera fuga.

Se probó además **como ataque**, no solo como comprobación: un usuario autenticado intentó dirigir el Spider al espacio de otro nombrándolo en el cuerpo de la petición, que era el quinto fallo del apartado 7. El espacio de la víctima quedó vacío y el rastreo fue a parar al del atacante.

Ni el código de invitación ni las contraseñas aparecen en ninguna salida: el guion de pruebas los mantiene en variables (R7).

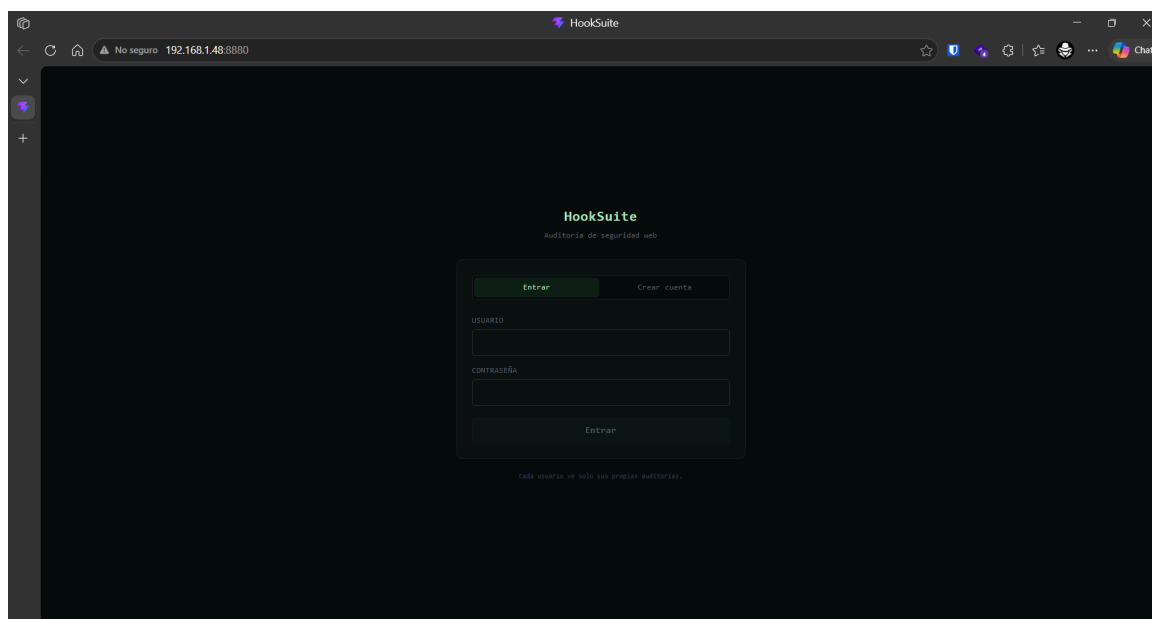


Figura 15: RF-12 — la entrada del producto tras retirar el Basic Auth de Nginx: un panel de acceso propio, con pestañas «Entrar» y «Crear cuenta». Antes, la raíz devolvía 401 pidiendo una credencial que ningún integrante del grupo conocía.

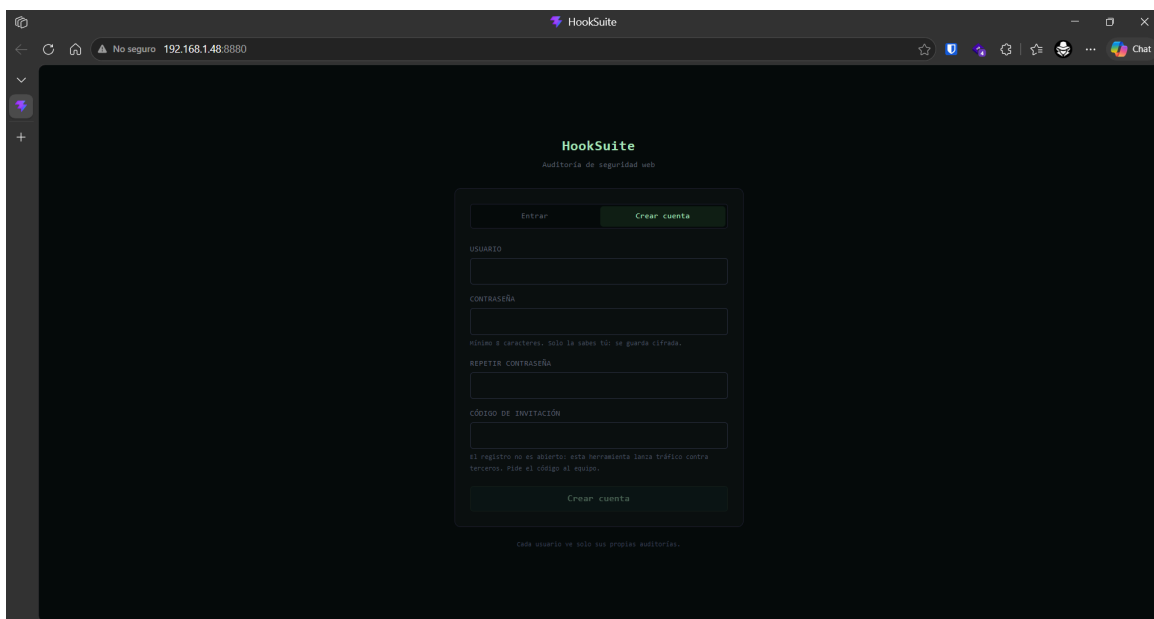


Figura 16: RF-12 — el alta exige **código de invitación**. El registro no es abierto a propósito: la herramienta lanza tráfico contra terceros, así que con altas anónimas cualquiera podría atacar desde la infraestructura del grupo. La contraseña la elige el usuario y solo la conoce él.

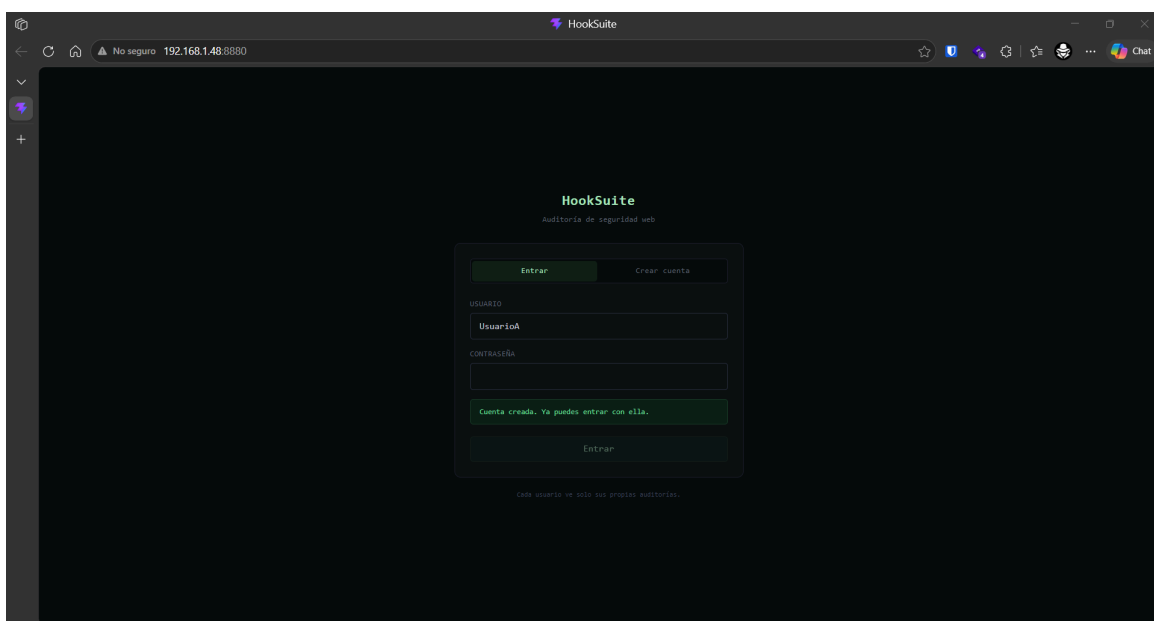


Figura 17: RF-12 — alta completada: la interfaz confirma la creación de la cuenta y devuelve al formulario de entrada. Cubre la mitad «registro» del requisito, que el enunciado pide junto al login.

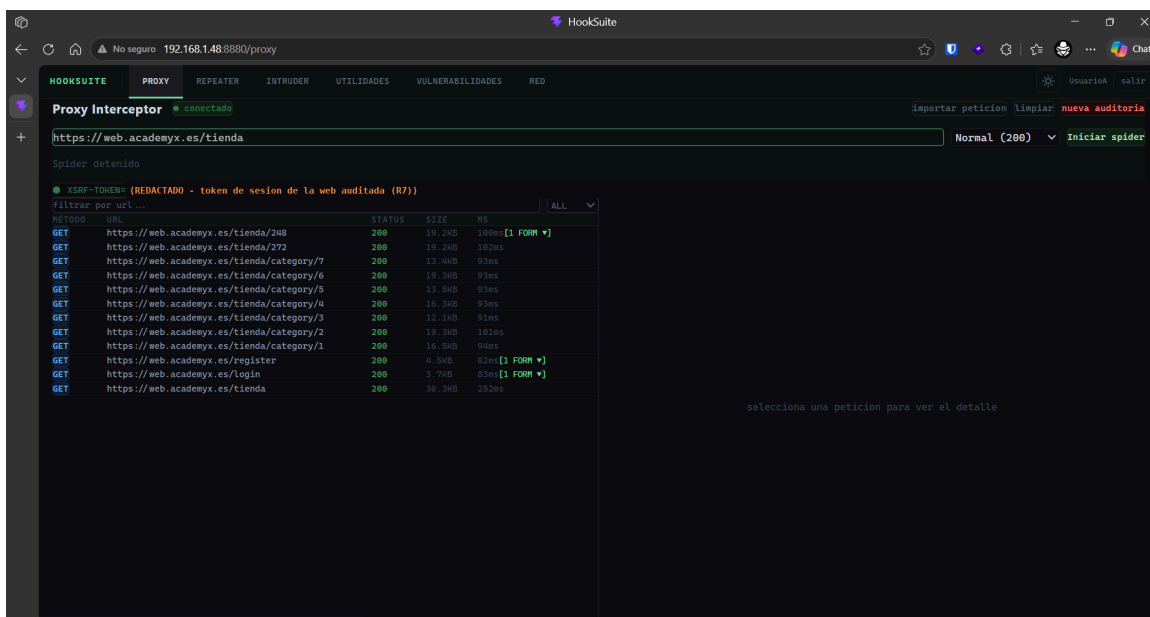


Figura 18: RF-12 — ANTES de la Fase 2 este panel era común a todos. Ahora el usuario A, autenticado, ve las trece peticiones que su propio Spider capturó del objetivo autorizado. Compárese con la captura siguiente, tomada al mismo tiempo.

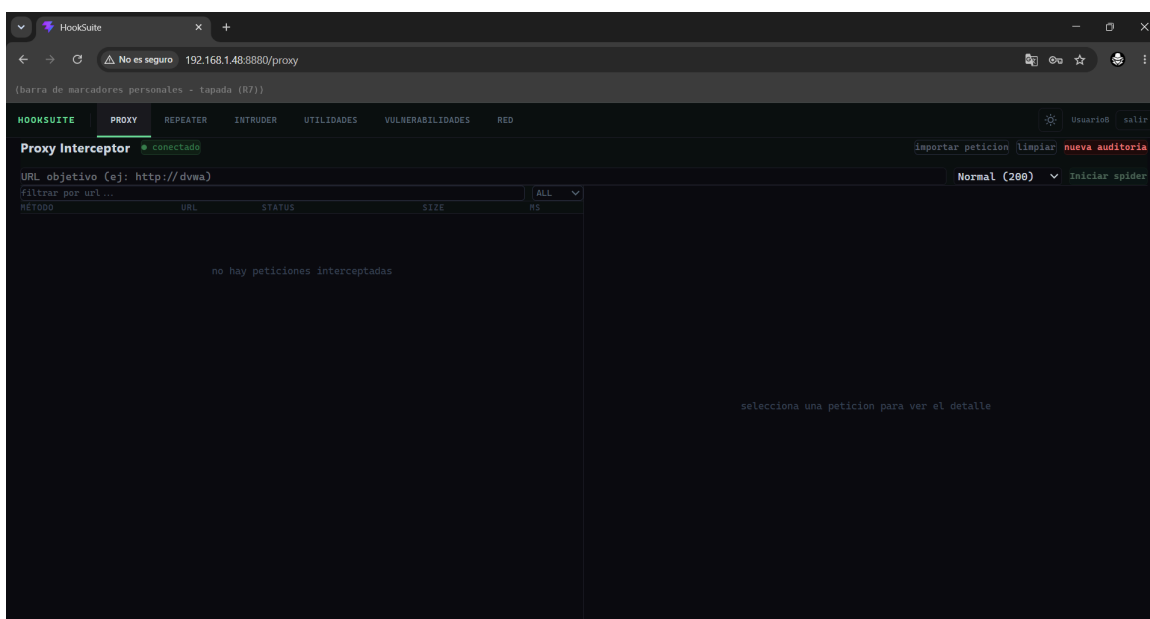


Figura 19: RF-12 — DESPUÉS, el contraste: el usuario B, autenticado a la vez y en otro navegador, ve su panel vacío. Antes de la Fase 2 habría visto exactamente lo mismo que el usuario A. Este par es la prueba del aislamiento por usuario.

> **Nota de tratamiento de las capturas (R7).** La del usuario A mostraba el valor íntegro de la cabecera XSRF-TOKEN, que es el identificador de sesión capturado de la web auditada: justo el dato que este requisito existe para no compartir. Se publica con ese valor tapado y la etiqueta a la vista, para que se vea qué se redactó; el original no sale del almacén interno. La del usuario B lleva tapada la barra de marcadores del navegador, ajena al proyecto.

Prueba preparada y NO ejecutada: el techo de gasto del módulo de IA (RF-08)

El plan de pruebas incluye un caso que **no se ha podido ejecutar**, y se declara en lugar de omitirse. Una auditoría del módulo de IA no tiene límite de consultas al modelo: el recorrido anida páginas × tipos de prueba × cargas, y cada carga es una consulta. Con el disparador de la interfaz construido, cada pulsación arrancarí­a ese recorrido completo.

El caso está diseñado para costar **cero**: fijando el techo en 0 por variable de entorno, ninguna consulta debe salir y la auditoría debe quedar marcada como troncada; con el techo en 1 debe pasar una sola y el

resto quedar como «no analizado», con el motivo del techo. Reutiliza la vía degradada de RNF-06, de modo que el recorte **se ve** en el panel en lugar de pasar por «sin hallazgos».

Por qué no está ejecutada: exige aplicar antes la reescritura del cliente del modelo, y esa reescritura no se aplica sin una clave de API válida con la que probarla de verdad (decisión del 8-oct: preparar, no ejecutar). El plan completo, con el código listo para aplicar, está en el repositorio de la línea.

⚠ **HUECO** — responsable: **José María + Claude**. Ejecutar el caso del techo de gasto en cuanto haya clave válida: captura de terminal con techo 0 y con techo 1, y captura de pantalla del panel mostrando la auditoría marcada como truncada. Coste previsto: 0 € la primera, una consulta la segunda.

AISLAMIENTO POR USUARIO VERIFICADO EN LA COCINA (11/11) · PENDIENTE DE VERIFICAR EN PRODUCCIÓN

EVIDENCIA DE EXPOSICIÓN Y DE LA FASE 1 COMPLETA

⚠ **HUECO** — responsable: **José María + Claude**. Plan de pruebas formal y capturas de PRODUCTO de los bugs (500 de `check/alive`, trampa del Spider) reproducidos en la cocina. Las de pantalla las saca José María (R6), tras la Fase 3, cuando el login nuevo cambie las pantallas. Las pruebas las hacen José María y Claude (R2).

Operación degradada (RNF-06), probada sin gastar saldo

La prueba que demuestra RNF-06 no necesita ni clave de API ni conexión: sustituye el cliente del modelo por uno simulado y comprueba qué devuelve el clasificador en cada situación. Ocho casos: los tres estados sobre la vía del proxy, el estado degradado en las cuatro vías, el camino correcto de la identificación de servidor, y que se le entrega un esquema de respuesta al cliente.

El valor de la prueba está en haberla visto fallar. Ejecutada contra el código anterior devuelve *0 de 8*, y el motivo es revelador: los cuatro métodos llamaban a `.get()` sobre el objeto que el cliente reescrito devuelve ahora, de modo que la fase de análisis no es que fallara en silencio, es que se interrumpía entera. Contra el código corregido devuelve *8 de 8*. Una prueba que pasa siempre no prueba nada; esta distingue las dos versiones.

Por qué una prueba y no una revisión visual. El fichero del clasificador está en la lista de material que este equipo no vuelca a la conversación con la herramienta de IA (apartado 12), así que se editó sin leerlo, mediante una receta de pasos verificables. En ese modo de trabajo la prueba no es un complemento de la revisión: es lo que la sustituye, y es más estricta, porque comprueba el comportamiento en vez de la impresión que deja el código al leerlo.

Reproducible con `python3 ia/tests/test_tres_estados.py`. Evidencia completa, con las dos salidas, en `docs/evidencias/rnf06-tres-estados-antes-y-despues-09oct.md`.

9. Matriz de trazabilidad

La pieza con la que se corrige la práctica: requisito por requisito, dónde está implementado, qué prueba lo verifica y dónde se ve. Las rutas están comprobadas en el código (cocina local, 4-oct). La columna de evidencia enlaza la figura de la memoria y el **minuto exacto del vídeo**; se rellena al grabar.

Req.	Descripción	Implementación (ruta)	Evidencia
RF-01	Dashboard web sin instalación	<code>PR-01/ src/ App.jsx + Layout.jsx</code>	fig. — · vídeo — —
RF-02	Modificado (2 errores de servidor corregidos en F1) del lado	<code>PR-02/ services/ proxy_service.py</code>	justif. apdo. 4 ·

	servidor (httpx)		apdo. 6
RF-03	Simple (4 defectos corregidos en F1) rastreo	PR-03 PR-03d/ services/ spider_service.py, frontend/.../ ProxyPage.jsx	figs. apdo. 7, . vídeo —: —
RF-04	Simple autenticada (sesión compartida)	PR-04 PR-04d/ services/ session_service.py	fig. — vídeo —: —
RF-05	Repeater (parser y avisos de error corregidos en F1)	PR-05 PR-05d/ routes/ repeater.py, backend/ services/ proxy_service.py	apdo. 6 . vídeo —: —
RF-06	Simple (cancelación corregida en F1) fuzzing	PR-06 PR-06d/ services/ intruder_service.py	apdo. 6 vídeo —: —
RF-07	Simple (incl. JWT)	PR-07 PR-07d/ src/ pages/ utilities/	fig. — . vídeo —: —
RF-08	Según alcance → panel Vulnerabilidades	PR-08 PR-08d/ analyzers/ vulnerability_classifier.py + backend/ routes/ vulnerabilities.py	fig. — vídeo —: —
RF-09	Según alcance → panel Red	PR-09 PR-09d/ tools/ + backend/ routes/ network.py	fig. — . vídeo —: —
RF-10	Según alcance	PR-10 PR-10d/ + backend/ routes/ playwright.py	fig. — . vídeo —: —
RF-11	Simple (reimplementado en F2) -usuario	PR-11 PR-11d/ services/ guardia.py + services/ auth_service.py	apdo. 7 . figs. apdo. 8
RF-12	Simple en F2 (verificado en cocina 11/11; pendiente de producción) y login por usuario	PR-12 PR-12d/ services/ auth_service.py, guardia.py, usuarios_store.py	apdo. 7 figs. apdo. 8

	+ aislamiento	frontend/.../LoginPage.jsx	
RNF-02	Verificación de token en tiempo real	PR-17d/main.py (/ws/)	apdo. 7 vídeo —
RNF-03	Despliegue Docker en Hetzner	PR-13-compose.yml + infra/nginx.conf	fig. — vídeo —
RNF-05	Control de concurrencia	PR-20d/services/intruder_service.py (asyncio.Semaphore)	apdo. 8
RNF-07	Seguridad verificada en F1 (exposición cerrada y servidor endurecido) y F2 (autenticación y aislamiento, verificados en cocina) producto	PR-14, PR-16 apartado 7	apdo. 7-8
RNF-08	Operativo y endurecido en F1 (permisos del canal y validación de entradas) por sesión	PR-15ll_agent.	apdo. 7-8
RNF-09	Protocolo HTTPS	PR-18 configurar en Nginx)	apdo. 8

RUTAS VERIFICADAS EN CÓDIGO · PRUEBAS Y MINUTOS DE VÍDEO POR COMPLETAR

> **Corrección declarada de este documento (7-oct), por R9.** La fila de RF-11 citaba como implementación la ruta `/api/session/new`. Esa ruta **se retiró en la Fase 2**: repartía identificadores que no eran credenciales, y la interfaz no la llamaba desde hacía tiempo. Citar como prueba de un requisito una ruta que ya no existe es el tipo de error que una memoria arrastra hasta la defensa, así que se deja dicho en lugar de corregirlo en silencio. Quien cumple hoy RF-11 es el sistema de acceso por usuario.

El estado de cada prueba (PR-xx) y su resultado detallado están en el apartado 8. Al arreglar cada fallo se reejecuta la prueba y se actualiza la fila. La numeración RF/RNF es la fijada en el apartado 4 y

PENDIENTE DE VALIDAR POR EL GRUPO

⚠ **HUECO** — responsable: **José María + Claude**. Al revisar el vídeo grabado, anotar el minuto exacto donde se demuestra cada requisito y el número de figura de su captura (R6: el nombre de la captura va atado al ID del requisito, p. ej. ``RF-08-panel-ia.png``, para que esta columna se rellene sola).

10. Limitaciones y trabajo futuro

Qué no funciona, qué funciona con condiciones y qué se haría con más tiempo. Este apartado recibe material hasta el último día.

El estado de sesión es volátil, por diseño

El estado de una sesión de auditoría —tráfico capturado, resultados del Intruder, vulnerabilidades— vive **en memoria del backend**: al reiniciarlo, se pierde. Es una **decisión consciente**, no un olvido, y se sostiene en tres razones:

1. **El patrón de acceso del backend no lo soporta sin reescribirlo.** Una treintena de puntos obtienen el diccionario de la sesión y lo mutan en memoria (`session["requests"].append(...)`). Persistir de

verdad obligaría a escritura-a-través en cada mutación, es decir, a tocar todo el backend a pocos días de la entrega.

2. **El modelo de sesión cambia en la Fase 2.** El login por usuario ata las sesiones a identidades; lo que se persistiera con el modelo anterior habría que rehacerlo.
3. **Una sesión de auditoría es un espacio de trabajo efímero.** El operador lanza un rastreo, examina los resultados y exporta lo que importa; no espera que la herramienta recuerde una sesión de hace tres días.

Lo que **sí** se hizo, porque era el riesgo real detrás de «todo en memoria»: **poner topes**. El recolector de sesiones existía en el código pero nunca se invocaba, y cualquier llamada a la API con un identificador nuevo creaba una sesión más —con la API sin autenticación, eso permitía agotar la memoria del servidor desde fuera—. Ahora hay un tope de sesiones y de peticiones por sesión (configurables por entorno), el recolector se ejecuta de verdad y, al desalojar, **prefiere las sesiones que no tienen conexión en tiempo real abierta**, para no interrumpir a quien está trabajando. Verificado con prueba de abuso: 120 identificadores inventados dejan el número de sesiones en el tope, y la sesión con conexión viva sobrevive.

Trabajo futuro. La persistencia es viable sin cambiar la arquitectura: el despliegue ya incluye Redis y el backend ya lo usa para otro flujo. El momento correcto es **después** del modelo de usuarios, para persistir sesiones ya asociadas a su propietario.

El coste de una auditoría con IA no está acotado

El módulo de IA no limita cuántas veces consulta al modelo durante una auditoría: el recorrido anida páginas × tipos de prueba × cargas, y cada carga es una consulta. Hoy es inocuo, porque el módulo no se puede disparar desde la interfaz; deja de serlo en el momento en que exista el disparador, que es trabajo en curso. El antecedente está en el apartado 3: en la P1, un disparo involuntario de este mismo recorrido agotó el saldo de la API.

Trabajo futuro, diseñado y no aplicado. Un techo de consultas por auditoría, configurable por entorno y situado en el **cliente** del modelo —por donde pasan las cuatro vías de análisis, de modo que ninguna se escapa—, que al agotarse devuelva la vía degradada de RNF-06 para que la auditoría quede marcada como truncada en vez de parecer completa. Un techo silencioso sería peor que no tenerlo: dejaría el informe diciendo «sin hallazgos» sobre una auditoría a medias.

El valor por defecto se fijará **con el dato** de la primera auditoría real —el resultado llevará el número de consultas consumidas— y no a ojo, que es lo que permite defenderlo.

 **HUECO** — responsable: **José María + Claude**. Acumular el resto de limitaciones conforme aparezcan durante las Fases 1-4; este apartado recibe material hasta el último día.

11. Reparto del trabajo

Quién hizo qué, con estimación de horas por persona.

 **HUECO** — responsable: **José María**. Consolidar las horas por persona al congelar (13-oct). El reparto real del trabajo queda trazado por los commits firmados por cada responsable (R2); José María cierra el recuento.

12. Uso de herramientas de IA

Declaración obligatoria de qué herramientas de IA se usaron y para qué.

- **Claude Code** (modelo Claude de Anthropic): asistencia en diagnóstico, montaje de la cocina, ensayo de limpieza del historial y redacción de esta memoria, bajo revisión humana.

Los límites de la herramienta, observados en primera persona

Usar un asistente de IA para auditar seguridad tiene un límite que conviene declarar porque lo encontramos trabajando: **el filtro de seguridad del propio modelo bloquea a veces tareas legítimas**

de este dominio. El 9 de octubre, leyendo las instrucciones del clasificador de vulnerabilidades de HookSuite, la petición fue rechazada con un error que lo dice con todas las letras —el filtro «a veces marca tareas legítimas de programación, **ciberseguridad** y biología»—. No es un fallo del proyecto ni del operador: es el coste de trabajar en seguridad ofensiva con una herramienta generalista, y conviene contarlo con la fecha y la referencia delante en lugar de omitirlo.

Lo caro no fue el bloqueo, sino **la recuperación**. Para desatascar la sesión se usó la función de rebobinado del cliente, eligiendo entre sus cuatro opciones la que descarta lo posterior al punto de retorno; se perdieron así ocho minutos de trabajo. Las otras dos opciones útiles —resumir desde el punto, o resumir solo lo anterior y permanecer al final de la conversación— habrían conservado ese trabajo. Es una lección de herramienta, no de ingeniería, pero cuesta horas igual.

De ahí sale el hallazgo que sí afecta al diseño de nuestros controles: **el rebobinado restaura la conversación y los ficheros, pero no los efectos sobre el servidor**. En la lista de puntos de retorno, el turno que había construido una imagen de contenedor figuraba como «sin cambios de código», porque esa función contabiliza ficheros. La imagen siguió existiendo después de rebobinar. Quien la construyó — el asistente— ya no recordaba haberlo hecho, y el único testigo del cambio fue un control externo que compara el estado del servidor contra lo que la documentación afirma. La consecuencia práctica es que **un control que mira ficheros no sustituye a uno que mira el sistema**, y que los avisos molestos de ese control no se pueden desactivar sin perder exactamente aquello para lo que sirve.

Qué fue generado y qué fue verificado. La distinción no es teórica: ese mismo día, una afirmación escrita por el asistente en el plan del módulo de IA —que los cuatro conjuntos de instrucciones pedían un campo `descripcion`— resultó falsa para uno de ellos al releer el código fuente, y quedó corregida de forma declarada en vez de reescrita en silencio. El criterio que seguimos es que **nada entra en esta memoria por haberlo dicho la herramienta**: entra cuando se ha contrastado contra el código, la salida de un comando o el enunciado.

 **HUECO** — responsable: **José María**. Detallar por fase qué aportó la IA y qué fue decisión/revisión humana; distinguir generado de verificado.

13. Anexos

Manual de instalación ampliado, credenciales de prueba (ficticias), glosario y referencias.

 **HUECO** — responsable: **José María**. Manual de despliegue, usuarios de prueba del login nuevo (sustituyen a la credencial de Nginx), glosario y referencias.